

# Arboles de Decision: Construyendo Intuicion

(Notebook: large\_trees\_exploration)

## Abstract

Este documento explica, paso a paso, como funcionan las estrategias de pooled testing usando arboles de decision. Empezamos con 2 personas y 1 test, y vamos subiendo hasta 8 personas con poblaciones heterogeneas. No se cubren aqui los parametros alpha/beta (ver documentos 02 y 03) ni las comparaciones con la tesis de Nico (ver documento 03).

## 1 Caso mas simple: 2 personas, 1 test

Tenemos 2 personas, cada una con probabilidad  $p = 0.2$  de estar infectada. Utilidad  $u = 1$  por persona aclarada. Solo 1 test disponible.

**Opcion A:** testear persona 0 sola.

- Si sale negativo (prob 0.8): aclaramos 1 persona.  $EU = 0.8$ .
- Si sale positivo: no aclaramos a nadie.

**Opcion B:** testear a las 2 juntas (pool).

- Si  $r = 0$  (prob  $0.8 \times 0.8 = 0.64$ ): aclaramos a las 2.  $EU = 0.64 \times 2 = 1.28$ .
- Si  $r > 0$ : no aclaramos a nadie.

**Resultado:** agrupar gana ( $EU\ 1.28$  vs  $0.80$ ) porque con baja prevalencia es probable que ambas esten sanas.

## 2 Efecto del valor: utilidades asimetricas

Misma configuracion pero ahora persona A vale  $u_A = 10$ , persona B vale  $u_B = 1$ . Ambas con  $p = 0.1$ .

Cuando una persona vale mucho mas, puede convenir testearla sola para no “desperdiciar” el test si la otra esta infectada.

Al barrer  $u_A$  de 1 a 20, hay un punto de cruce donde individual empieza a ganarle al pool.

## 3 Efecto de la prevalencia

$n = 3$ ,  $B = 2$ ,  $G = 3$ . Tres personas iguales ( $u = 1$ ), variamos  $p$ :

- $p = 0.05$  (baja): conviene pool grande. Casi seguro todos sanos.
- $p = 0.20$  (media): pool sigue ganando, pero con menos margen.
- $p = 0.50$  (alta): pools chicos o individuales. Muchos positivos.

**Regla general:** baja prevalencia  $\rightarrow$  pools grandes. Alta prevalencia  $\rightarrow$  pools chicos.

## 4 Aumentado vs Clasico

$n = 3$ ,  $B = 2$ ,  $G = 3$ , probabilidades  $p = [0.2, 0.25, 0.15]$ , utilidades  $u = [3, 2, 4]$ .

- **Test clasico:** solo dice positivo/negativo (binario).
- **Test aumentado:** dice *cuantos* infectados hay ( $r = 0, 1, 2, \dots$ ).

El aumentado da estrictamente mas informacion. Permite mejores actualizaciones bayesianas despues de un resultado positivo. Por ejemplo, si  $r = 1$  en un pool de 3, sabemos que exactamente 1 esta infectado. El clasico solo sabe que “al menos 1” lo esta.

**Resultado:** aumentado siempre  $\geq$  clasico en utilidad esperada.

## 5 Poder del presupuesto

$n = 4$ ,  $G = 4$ ,  $p = 0.15$ ,  $u = 1$ . Variamos  $B = 1, 2, 3, 4$ .

Mas tests = mas oportunidades de aclarar personas. La utilidad esperada sube con  $B$  pero con rendimientos decrecientes: cada test adicional aporta menos que el anterior. Con suficientes tests, nos acercamos a  $U_{\max}$  (la cota superior si supieramos quien esta infectado).

## 6 Por que no testear uno por uno?

$n = 4$ ,  $B = 2$ ,  $p = 0.15$ ,  $u = 1$ . Variamos  $G = 1, 2, 3, 4$ .

$G = 1$  obliga a tests individuales: con  $B = 2$  solo aclaramos 2 personas.  $G = 4$  permite meter a las 4 en un pool: si sale  $r = 0$ , aclaramos a las 4 de golpe.

**Resultado:** a mayor  $G$ , mejor EU (cuando la prevalencia es baja). Agrupar multiplica el impacto de cada test.

## 7 Poblacion heterogenea: VIPs vs comunes

$n = 8$ ,  $B = 3$ ,  $G = 3$ .

- 4 VIPs:  $p = 0.3$ ,  $u = 10$  (alto valor, moderada infeccion).
- 4 comunes:  $p = 0.1$ ,  $u = 1$  (bajo valor, baja infeccion).

El optimizador (DP) prioriza aclarar a los VIPs porque valen 10x mas. El greedy puede no hacerlo bien porque es miope (solo ve un paso adelante).

Al variar  $p_{VIP}$  de 0.1 a 0.5, el gap greedy-optimo crece: a mayor heterogeneidad, mas importa optimizar bien.

## 8 Busqueda binaria como heuristica

$n = 6$ ,  $B = 4$ ,  $G = 3$ ,  $p = 0.15$ ,  $u = 10$ .

La busqueda binaria es una heuristica natural: testea la mitad, si sale positivo subdivide. Pero el DP optimo es mas flexible — puede elegir pools que no son mitades exactas, explotando la informacion del conteo  $r$ .

Comparacion:

- DP optimo: mejor EU.
- Greedy: casi tan bueno como DP.
- Búsqueda binaria: peor que ambos (no usa info de conteo).

## 9 Greedy vs Optimo: cuanto se pierde?

Grid de instancias aleatorias:  $n \in \{4, 5, 6\}$ ,  $B \in \{2, 3\}$ ,  $G \in \{3, 4\}$ .

El gap greedy–optimo varia por instancia (0–15%). **Greedy counting** (usa toda la historia bayesiana) supera a **greedy secuencial** (actualiza test por test).

## 10 Solver hibrido: K pasos greedy + DP

$n = 6$ ,  $B = 4$ ,  $G = 3$ . Variamos  $K$  de 0 (todo DP) a  $B$  (todo greedy).

- $K = 0$ : solucion exacta, pero lenta.
- $K = B$ : rapido, pero pierde calidad.
- $K = 2-3$ : recupera 80–90% de la calidad del DP con una fraccion del tiempo.

Este es el enfoque practico: usar greedy para las primeras decisiones (donde el espacio es grande) y DP para las ultimas (donde el espacio ya se redujo).